

What are we after? – association of pni_if with a better description
A better description of the physical network interface (PNI).

```
#!/bin/sh

pni_if=$(ls /sys/class/net)

# Capture once to a variable to avoid multiple sudo calls and for speed
# We use -json because it ensures fields are on predictable lines
raw_json=$(sudo lshw -json -C network)

for pni in $pni_if; do
    if [ "$pni" = "lo" ]; then
        echo "lo: Loopback (127.0.0.0/8) - Internal"
    else
        # Use awk to find the specific block where logicalname matches $pni
        # RS='}' treats every JSON object as one record
        # FS='"' uses double quotes as field delimiters for easy value
        extraction

        info=$(echo "$raw_json" | awk -v target="$pni" '
            BEGIN { RS="}"; FS="\"" }
            $0 ~ "\"logicalname\" : \""target\"" {
                # Loop through the record to find our keys
                for (i=1; i<=NF; i++) {
                    if ($i == "vendor") vendor = $(i+2)
                    if ($i == "product") product = $(i+2)
                    if ($i == "description") desc = $(i+2)
                }
                if (product != "") {
                    printf "%s | %s | %s", vendor, product, desc
                }
            }
        ')

        if [ -n "$info" ]; then
            echo "$pni: $info"
        else
            # Handle virtual interfaces (docker, veth, etc) that lshw ignores
            echo "$pni: Virtual or Software Interface"
        fi
    fi
done
```

-- awk power to make sure every PNI is correctly associated with:

- (a) product
- (b) vendor
- (c) description

Consider multiple network interfaces. Yes, there are many types of computers that do have more than one PNI(wire/wireless); https://github.com/ahrink/gvf/blob/pdf/pni_essay.pdf

Why this is "Stronger"

- **Encapsulation:** By setting RS="}", we ensure awk doesn't accidentally pull a vendor from "Interface A" and associate it with "Interface B." It looks at one hardware block at a time.

- **Field Splitting:** By using `FS="\\"`, the value of a JSON key like `"vendor" : "Intel Corp"` is always exactly 2 fields away. This is much more robust than using `grep` or `cut` on column numbers.
- **No Multi-Sudo:** It runs `lshw` once, stores the result, and processes it in memory. This is significantly faster on systems with many interfaces.
- **Pure POSIX:** This will run on `dash`, `ash`, or `busybox` without needing any modern Linux extensions.

Quick Formatting Note

In JSON, the key/value pairs can sometimes appear in different orders. The `for (i=1; i<=NF; i++)` loop inside the `awk` block ensures that even if `lshw` changes the order of "vendor" and "product" in a future update, GVF/AHR script will still find them.

TODO: is to adjust how it handles empty fields (e.g., if a vendor is missing but a product exists) ...

REM: Use function to prevent no-PNI since most cases are one or many PNI/s (blocks).

San Anton 20260221 via AHR "PNI is getting better, GVF?"

PS: discussion based on the output-test as needed due to the infancy of the script not because it doesn't work;

```
# typical laptop single interface block split as wired and wireless cases - same
vendor and product applies to both.
# running on Ubuntu 24.04 (kubuntu)
enx00e0xxxxxc4: Virtual or Software Interface
lo: Loopback (127.0.0.0/8) - Internal
wlp0xxxf3: Intel Corporation | Ice Lake-LP PCH CNVi WiFi | Wireless interface

# typical multi PNI in server or workstations - dual wired interface-block;
# running on Ubuntu 22.04 server
eno0: Intel Corporation | 82599ES 10-Gigabit SFI/SFP+ Network Connection |
Ethernet interface
enp17xxxxxp0: Solarflare Communications | XtremeScale SFC9250 10/25/40/50/100G
Ethernet Controller | Ethernet interface
enp17xxxxxp1: Solarflare Communications | XtremeScale SFC9250 10/25/40/50/100G
Ethernet Controller | Ethernet interface
enp24s0f1: Intel Corporation | 82599ES 10-Gigabit SFI/SFP+ Network Connection |
Ethernet interface
lo: Loopback (127.0.0.0/8) - Internal
```

1. (read the pdf pls) ...
2. it means that the interface-block was captured correctly.
3. note the new paradigm for Predictable Network Interface (another PNI) and instead of "enx00e0xxxxxc4: Virtual or Software Interface" should have been "enx00e04c6802c4: Predictable Intel Corporation | Ice Lake | wired interface" – that is a minor cosmetic.
4. "Interface A" and associate it with "Interface B." – there is a misconception, in that, an interface block can be both wired and wireless vs other interface block can be both dual-wired but still the same block.
5. No-PNI has two cases RE: interface block
 - (a) PNI cannot be detected due to malfunction or requires a specific driver that the OS is not supporting.

(b) The computer does NOT have a PNI [:0)]